

What We Each Build & What B1 Looks Like When Done

4 members · 2-week sprint · Intelligent Traffic System

What Each Person Builds

HARDWARE ENGINEER Cameras, Pi 5 & Physical Setup

You are responsible for the physical layer. Everyone else depends on you — if the cameras are not running, nothing works.

- **Set up the Raspberry Pi 5** — install Balena OS, register it in BalenaCloud
- **Connect and verify all cameras** — USB cameras via /dev/videoN, IP cameras via RTSP URL
- **Create lanes.json for each camera** — draw lane polygons using calibrate.py, measure pixel-to-metre ratio
- **Test camera disconnects** — unplug each camera, confirm the container retries every 5s and others keep running
- **Run the junction for 10 minutes** — all cameras live simultaneously, no crashes

→ **Your output:** *Pi 5 online in Balena, all cameras streaming, one lanes.json per camera, test evidence.*

ML / AI ENGINEER Detection, Tracking, Kafka & Buffer

You build the brain of B1. Raw frames come in, structured vehicle events go out. You also own the Kafka producer and offline buffer because those are tightly coupled to the event format you define.

- **Frame capture** — read from USB or IP camera using OpenCV, apply FRAME_SKIP
- **YOLOv8n detection** — output bbox, centroid, class (car/bus/truck/motorcycle/bicycle/pedestrian), confidence
- **ByteTrack tracking** — assign stable vehicle_id across frames
- **Speed estimator** — calculate km/h from centroid movement between frames
- **Kafka producer** — publish JSON events to traffic.events.raw, acks=all
- **SQLite offline buffer** — save events to disk when Kafka is down, replay when it comes back
- **Accuracy test** — run on test video, confirm class labels are correct ≥85% of the time
- **Sample .jsonl file** — 5 minutes of real detection events, send to L2 team by Day 7

→ **Your output:** *Working detection pipeline, events flowing into Kafka, buffer tested, .jsonl file delivered to L2.*

SOFTWARE ENGINEER Enrichment, APIs & Integration

You connect everything together and make B1 observable. The lane enricher feeds lane data back into the ML pipeline. The health and metrics endpoints let L4 see inside B1. Your integration tests prove the full layer works end-to-end.

- **Lane ROI enricher** — read lanes.json, map each detection centroid to a lane, attach lane_id
- **RTSP fallback server** — expose raw video via GStreamer so L2 can run its own detection if needed
- **Multi-camera isolation** — one container per camera — a crash in one does not stop the others
- **/health endpoint** — return HTTP 200 only when camera is live AND Kafka producer is connected
- **Prometheus /metrics endpoint** — expose frame rate, detections per second, Kafka publish rate, buffer depth
- **docker-compose.yml** — local 2-camera test setup for the team to run offline

- **Integration tests** — verify AC-01 (latency), AC-04 (Kafka outage), AC-05 (disconnect), AC-08 (L2 confirm)

→ **Your output:** *Lane enricher, /health, /metrics, RTSP fallback, docker-compose.yml, and passing integration tests.*

DEVOPS ENGINEER **Docker, Balena, CI/CD & Platform**

You make sure everything runs in production and that L4 can see and manage it. Without your work, the code never reaches the Pi and L4 has no visibility.

- **Dockerfile** — package the full Python pipeline — YOLOv8n, OpenCV, Kafka, GStreamer
- **Balena fleet config** — set all env vars, store camera and Kafka credentials as Balena Secrets
- **OTA update tested** — push a change via BalenaCloud, confirm Pi 5 updates without SSH
- **Kafka broker confirmed** — get host:port from L4, set KAFKA_BROKERS in fleet variables
- **CI/CD pipeline** — GitHub Actions — test → build → push image → Balena deploy on every merge to main
- **L4 sign-off** — written confirmation from L4 that /health, /metrics, Kafka, and containers are all accepted

→ **Your output:** *Dockerfile, Balena fleet running OTA, CI/CD pipeline live, and L4 written sign-off.*

What B1 Looks Like When We Are All Done

When all four of us have delivered, B1 runs like this from power-on to data in L2:

1 Pi 5 boots

Balena OS starts. The supervisor pulls the latest Docker image and launches one container per camera automatically. No manual setup needed.

2 Cameras come online

Each container opens its assigned camera — USB or IP. If a camera fails, that container retries every 5 seconds. The other cameras keep running unaffected.

3 Frames are captured and processed

OpenCV samples frames at the configured rate. YOLOv8n detects every vehicle and labels it (car, bus, truck, etc.). ByteTrack gives each vehicle a stable ID across frames.

4 Each detection is enriched

The lane enricher maps where the vehicle is (which lane). The speed estimator calculates how fast it is moving. Both values are added to the event.

5 Events are published to Kafka

A complete JSON event — camera ID, timestamp, vehicle ID, class, speed, lane — is sent to the Kafka topic `traffic.events.raw` within 500ms of the vehicle appearing on camera.

6 Outages are handled automatically

If Kafka goes down, events are saved locally in SQLite. When Kafka comes back, they are replayed in order. No data is lost and no one needs to intervene.

7 L2 receives the data

The L2 AI layer consumes events from Kafka, runs congestion analysis, and serves results to the dashboard. B1's job ends the moment the event lands in Kafka.

8 L4 monitors everything

L4 scrapes `/metrics` every 15 seconds. The `/health` endpoint tells L4 whether each camera and Kafka connection is alive. Alerts fire if anything drops.

9 Updates deploy with no SSH

When we merge code to main, GitHub Actions builds a new image and Balena delivers it to the Pi 5 over the air. The device updates itself.

We are done when all of these pass

#	What We Check	Owner
1	Events reach Kafka within 500ms of a vehicle appearing on camera	ML/AI
2	The same vehicle keeps the same ID for at least 10 frames	ML/AI
3	Vehicle class is correct 85% of the time on our test video	ML/AI
4	Events are saved and replayed correctly after a 60-second Kafka outage	ML/AI
5	Unplugging a camera does not crash the other containers	Hardware
6	A Balena OTA update reaches the Pi without any SSH access	DevOps
7	L4 Prometheus can scrape our <code>/metrics</code> endpoint	DevOps
8	L2 confirms they can consume our events and see congestion data	Software
9	Sample .jsonl file delivered to L2 by Day 7	Software

✓ B1 is complete

The Pi 5 is running at the junction, detecting every vehicle, publishing events to Kafka, surviving failures

on its own, and giving L4 full visibility. L2 can build everything it needs on top of what we produce.